



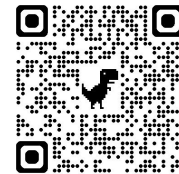
Adapter, Facade e Bridge

Padrões de Projeto - Aula #02

Prof. Dr. Carlos Melo

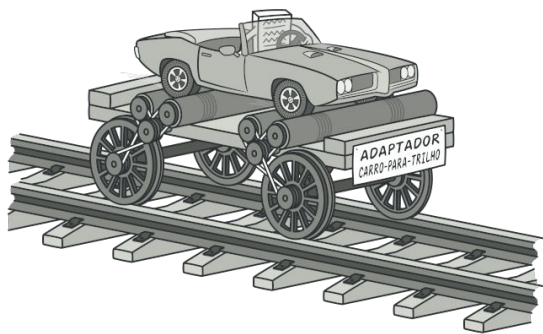
 carlos.melo@upe.br

Padrões Estruturais - #01



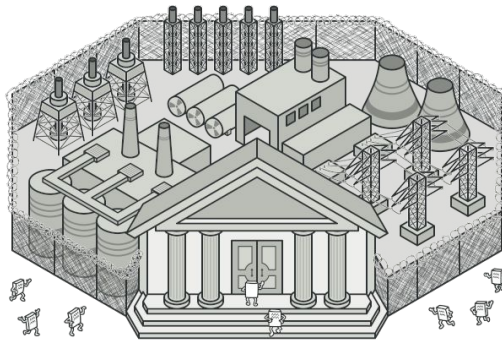
Adapter

Faz um carro andar em trilhos ao converter uma interface em outra



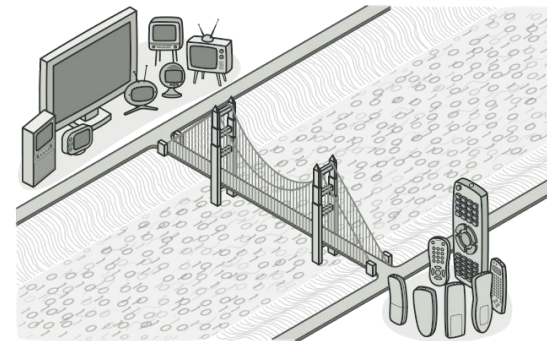
Facade

Uma porta de entrada única para um subsistema complexo



Bridge

Dois lados que variam de forma independente conectados por uma ponte

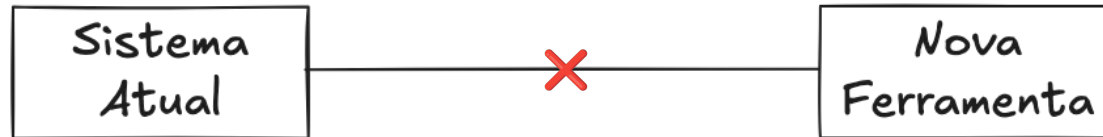


- Padrões estruturais tratam da composição de classes e objetos em estruturas maiores mantendo a flexibilidade
- Funcionam em cima de dois princípios:
 - programe para interfaces;
 - favoreça composição sobre herança.



Adapter - O Problema

- Sua equipe desenvolveu um sistema de análise de relatórios.
- Durante meses tudo funcionou perfeitamente.
- Até que a empresa comprou uma nova ferramenta de B.I.
- Agora precisamos integrar essa ferramenta ao sistema existente.



- O que normalmente fazemos quando duas APIs não conversam?



Adapter - Interfaces Incompatíveis



```
1 [  
2   {  
3     "id": 1337,  
4     "date": "2025-03-01",  
5     "final_price": 2350.50  
6   }  
7 ]
```

Nosso sistema espera



```
1 headers = [  
2   "id",  
3   "date",  
4   "final_price"  
5 ]  
6  
7 rows = [  
8   [1337, "2025-03-01", 2350.50]  
9 ]
```

Nova ferramenta entrega

- Mesma informação. Formatos incompatíveis.

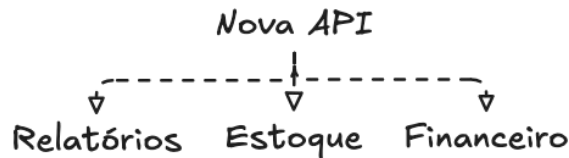


Adapter - Solução Ingênua

- Que tal modificar a aplicação para entender o novo formato?

```
1 def average():  
2     data = loader.rows  
3     headers = loader.headers  
4     ...
```

- Altera código que já funciona;
- Aumenta acoplamento;
- Vários módulos precisam conhecer a nova API;
- Qualquer mudança futura exigirá novas modificações.





Adapter - A ideia

- Em vez de mudar todo o sistema podemos **adaptar** um ponto de comunicação



"Converter a interface de uma classe em outra interface esperada pelos clientes."

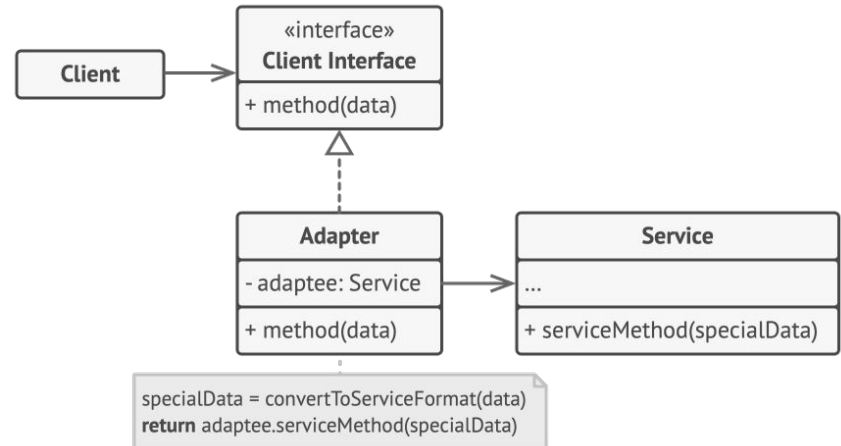
O Adapter permite que classes com interfaces incompatíveis trabalhem juntas."

- Também chamado de **Wrapper** (mesmo apelido do Decorator)
- O adaptador não altera nem o cliente nem a classe adaptada. Ele traduz.



Adapter - Estrutura

1. O **Client** é uma classe que contém a lógica de negócio do programa existente
2. A **ClientInterface** descreve um protocolo que outras classes devem seguir para colaborar com o código do cliente
3. A **Service** é alguma classe útil (geralmente de terceiros ou código legado) que é incompatível
4. O **Adapter** é uma classe que é capaz de trabalhar tanto com o cliente quanto com o serviço. O Adapter recebe chamadas do cliente e as traduz em chamadas para o objeto do serviço em um formato que ele entenda.





Winamp - Tempos áureos da Internet

- Nossa empresa é dona de um reprodutor de mídia... mas ele só reproduz áudio.
- Sabemos programar: todas as classes seguem a mesma interface, então a comunicação entre elas ocorre sem dificuldades.

```
1 from abc import ABC, abstractmethod
2
3 class Target(ABC):
4     @abstractmethod
5     def play(self): ...
6
7 class AudioPlayer(Target):
8     def play(self):
9         print("Reproduzindo áudio")
10
11 class MediaPlayer:
12     def __init__(self, player: Target):
13         self.player = player
14
15     def execute(self):
16         self.player.play()
```




Adapter - Winamp

- Queremos estender o player para reproduzir vídeo.
- Encontramos uma biblioteca pronta:

```
1 class VideoPlayer: # código de terceiros, não podemos alterar
2     def play_mp4(self):
3         print("Reproduzindo vídeo MP4")
```

- > MediaPlayer(VideoPlayer()) Vai funcionar? Vai
- Mas MediaPlayer(VideoPlayer()).execute() vai falhar, porque VideoPlayer não tem play() possui play_mp4()
- Interfaces incompatíveis.



Adapter - Winamp

- Em vez de alterar nosso sistema ou a biblioteca de terceiros, criamos uma nova classe:

```
1 class VideoAdapter(Target):  
2     def __init__(self, adaptee):  
3         self.adaptee = adaptee  
4  
5     def play(self):  
6         self.adaptee.play_mp4()
```

- O adaptador faz a "tradução" de play() para play_mp4().
- Precisamos alterar mais alguma coisa no sistema?
 - Não!
- Apenas o VideoAdapter conhece o VideoPlayer.
- Se a biblioteca mudar, um arquivo muda.

Winamp



```
1 MediaPlayer(AudioPlayer()).execute()  
2 # Reproduzindo áudio  
3 MediaPlayer(VideoAdapter(VideoPlayer())).execute()  
4 # Reproduzindo vídeo MP4
```

- E se depois quisermos reproduzir vídeos em outro formato?
 - Basta criar outro Adapter: MkvAdapter, AviAdapter.
- MediaPlayer não muda.



Prós e Contras do Adapter

Prós

- **S** do SOLID: a conversão de interface ou de dados fica separada da lógica de negócio
- **O** do SOLID: novos adaptadores entram sem quebrar o código cliente existente
- Isola o sistema de mudanças na biblioteca de terceiros: um único ponto de contato
- Permite integrar código que você não pode alterar (proprietário, legado, binário).

Contras

- Mais uma classe e mais um nível de **indireção** (número de vezes que preciso navegar por intermediários até chegar ao que quero)
- Às vezes é mais simples mudar a classe de serviço para que ela se adeque ao resto do código
- Muitos adaptadores empilhados escondem a estrutura real do sistema





Exercício

- Sua equipe desenvolveu um sistema que analisa relatórios financeiros.
- Ele recebe os dados por uma interface padronizada (DataLoader) que retorna uma lista de dicionários.
- A empresa adquiriu uma ferramenta de B.I. chamada BeeAI.
- Ela disponibiliza todas as informações necessárias, mas sua API usa um formato completamente diferente (headers + rows).
- O código da ferramenta é proprietário: não pode ser modificado.
- O sistema atual está em produção: não deve ser alterado.
- Desafio: integrar as duas aplicações sem modificar nenhuma delas.

Exercício



```
1 from abc import ABC, abstractmethod
2
3 class DataLoader(ABC):
4     @abstractmethod
5     def load_data(self) -> list[dict]: ...
```

data_loader.py (Target)

```
1 class ReportAnalyzer:
2     def __init__(self, loader: DataLoader):
3         self.loader = loader
4
5     def average(self) -> float:
6         data = self.loader.load_data()
7         total = sum(order["final_price"] for order in data)
8         return total / len(data)
```

report_analyzer.py (Client)

```
1 class BeeAiClient:
2     def __init__(self):
3         self.headers = ["id", "date", "final_price"]
4         self.rows = [
5             [1337, "2026-08-01", 2350.50],
6             [1338, "2026-08-02", 4800.50],
7             [1339, "2026-08-03", 1500.00],
8         ]
```

bee_ai.py (Ferramenta)

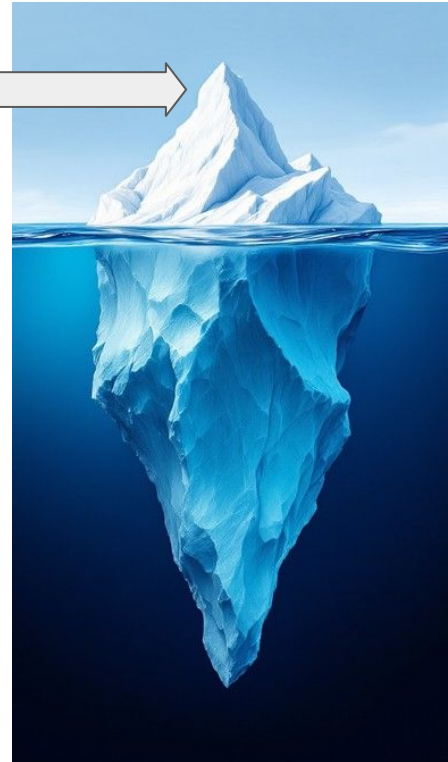


```
1 class BeeAiAdapter(DataLoader):
2     pass
3
4
5 cliente = BeeAiClient()
6 adapter = BeeAiAdapter(cliente)
7 analyzer = ReportAnalyzer(adapter)
8 print(analyzer.average())
```



Facade - O Problema

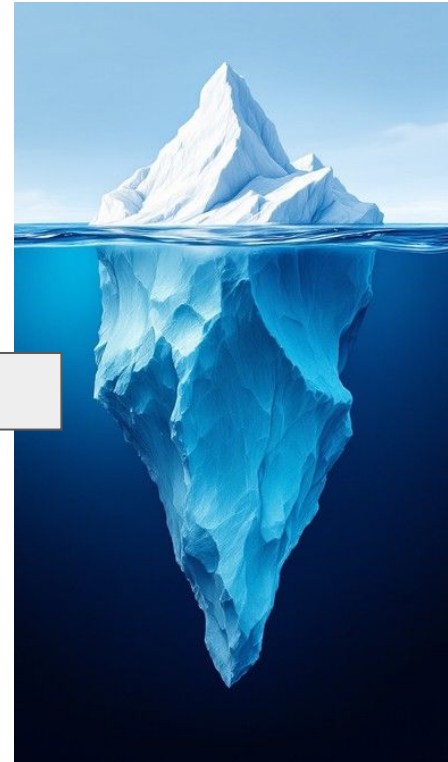
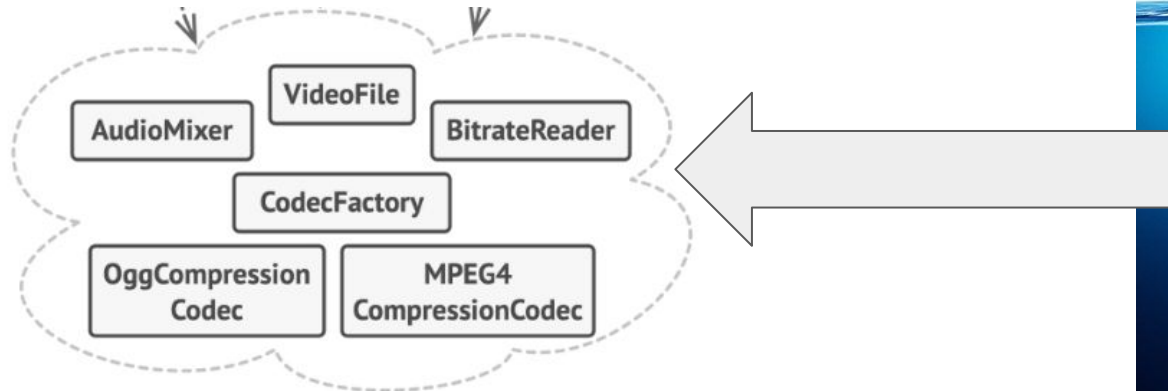
- Converter um vídeo parece simples, como usuário a gente espera algo do tipo:
- > `converter.convert("video.mov", "video.mp4")`





Facade - O Problema

- Converter o vídeo envolve diversos componentes especializados:
 - ler o arquivo de entrada;
 - identificar o codec utilizado;
 - decodificar o vídeo;
 - converter para o novo formato;
 - codificar novamente;
 - gravar o arquivo de saída.



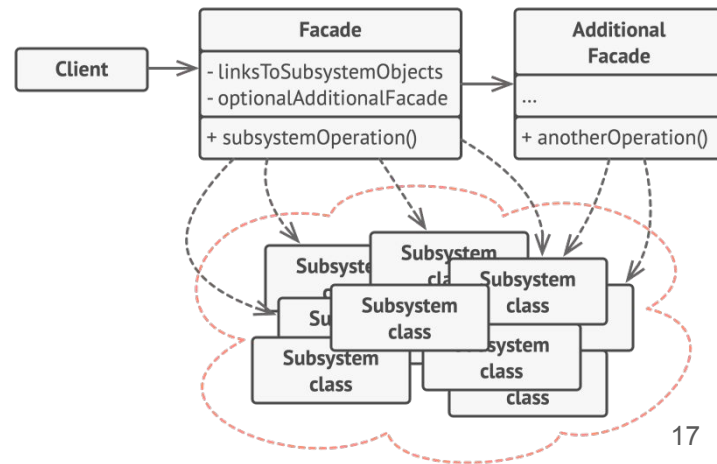


Facade - A ideia

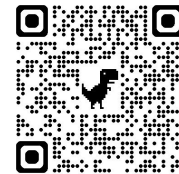
"Fornecer uma interface unificada para um conjunto de interfaces em um subsistema.

O Facade define uma interface de nível mais alto, que torna o subsistema mais fácil de usar."

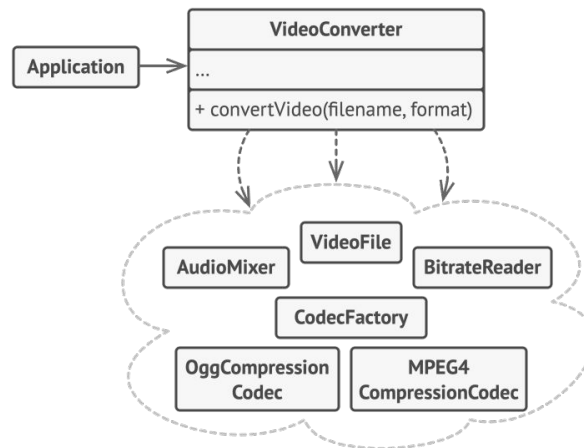
- Quem usa esse subsistema precisa conhecer todas essas classes?
- Se sim, o cliente fica acoplado a 6 classes para realizar uma única operação e a qualquer mudança interna do subsistema.
- Uma saída:
 - Oferecer um ponto de entrada único e simplificado para o caso de uso comum. Esse ponto de entrada é o Facade.



Facade - O sistema



```
1 class VideoFile:
2     def __init__(self, nome: str):
3         self.nome = nome
4         self.codec_name = nome.split(".")[1]
5         print(f"[VideoFile] arquivo {nome} aberto")
6
7 class OggCompressionCodec:
8     tipo = "ogg"
9
10 class MPEG4CompressionCodec:
11     tipo = "mp4"
12
13 class CodecFactory:
14     @staticmethod
15     def extrair(arquivo: VideoFile):
16         if arquivo.codec_name == "mp4":
17             print("[CodecFactory] codec MPEG4 extraído")
18             return MPEG4CompressionCodec()
19         print("[CodecFactory] codec OGG extraído")
20         return OggCompressionCodec()
21
22 class BitrateReader:
23     @staticmethod
24     def ler(arquivo, codec):
25         print(f"[BitrateReader] lendo com codec {codec.tipo}")
26         return b"<buffer>"
27
28     @staticmethod
29     def converter(buffer, codec):
30         print(f"[BitrateReader] convertendo para {codec.tipo}")
31         return b"<buffer convertido>"
32
33 class AudioMixer:
34     @staticmethod
35     def corrigir(resultado):
36         print("[AudioMixer] faixa de áudio corrigida")
37         return resultado
```



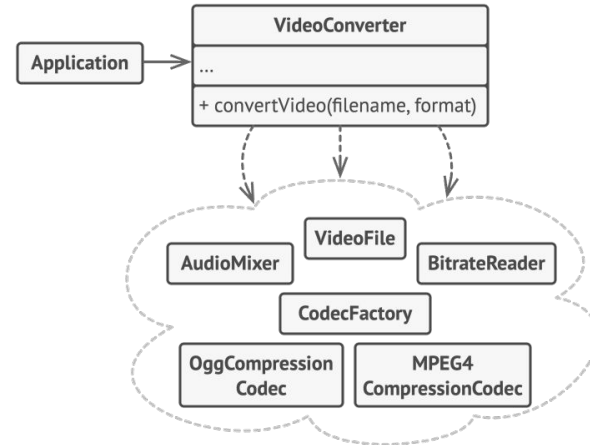


Facade - A fachada

```
1 class VideoConverter:
2     """Facade: um método para o caso de uso comum."""
3
4     def convert(self, nome_arquivo: str, formato: str) -> bytes:
5         arquivo = VideoFile(nome_arquivo)
6         codec_origem = CodecFactory.extrair(arquivo)
7
8         codec_destino = (MPEG4CompressionCodec() if formato == "mp4"
9                          else OggCompressionCodec())
10
11         buffer = BitrateReader.ler(arquivo, codec_origem)
12         resultado = BitrateReader.converter(buffer, codec_destino)
13         return AudioMixer.corrigir(resultado)
```

```
1 convertido = VideoConverter().convert("ferias.ogg", "mp4")
```

```
1 [VideoFile] arquivo ferias.ogg aberto
2 [CodecFactory] codec OGG extraído
3 [BitrateReader] lendo com codec ogg
4 [BitrateReader] convertendo para mp4
5 [AudioMixer] faixa de áudio corrigida
```





Prós e Contras do Facade

Prós

- Isola o cliente da complexidade de um subsistema.
- Reduz o número de dependências: 1 em vez de 6.
- Mudanças internas no subsistema não propagam para o cliente.

Contras

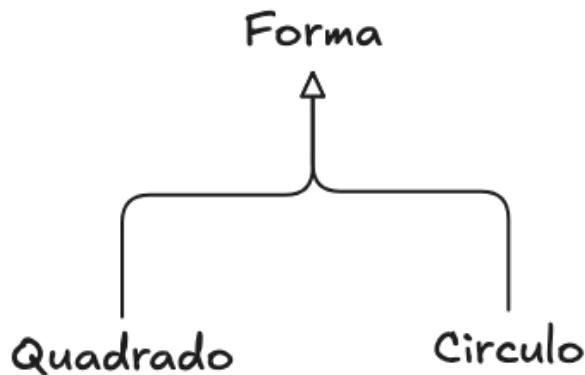
- A fachada pode virar um objeto-deus, acoplado a todas as classes da aplicação.
 - Mitigação: várias fachadas pequenas por caso de uso, em vez de uma gigante.
- Esconder a complexidade não a elimina, o subsistema continua lá.





Bridge - O Problema

- Estamos desenvolvendo uma biblioteca para desenhar formas geométricas.
- Inicialmente existem apenas duas formas:
 - Quadrado e Círculo

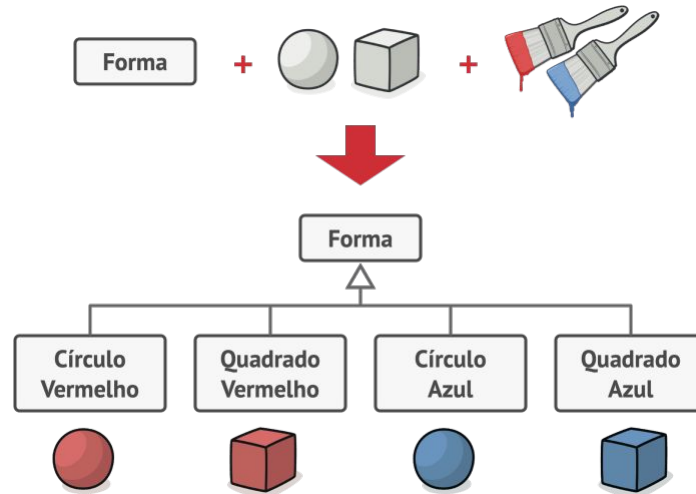


- Agora o cliente nos faz uma nova solicitação:
 - “Gostaria de desenhar essas formas em diferentes cores”



Bridge - Explosiva

- Uma subclasse por combinação.



- Essa solução resolve o problema?



Bridge - Explosiva

- E se precisarmos de um triângulo... quantas novas classes precisaríamos criar?
- E adicionar a cor verde?
- No final temos Forma x Cor
- Será que forma e cor deveriam aparecer numa mesma hierarquia?
- Será que herança é a melhor solução para o nosso problema?

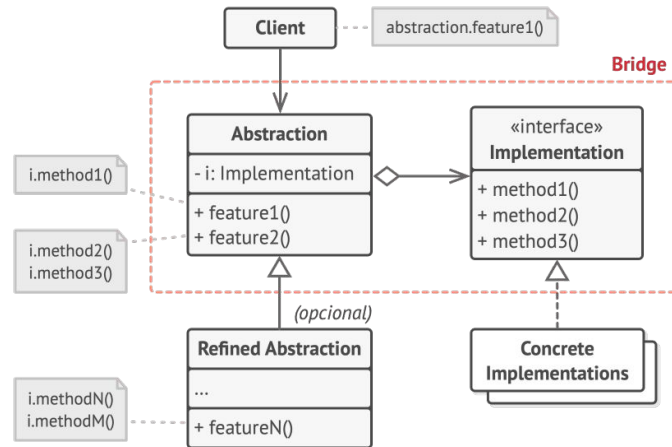
Cenário	Herança	Bridge
2 formas e 2 cores	4	4
+ triângulo	6	$3 + 2 = 5$
+ verde	9	$3 + 3 = 6$
4 formas e 5 cores	20	$4 + 5 = 9$

Herança cobra $m \times n$ e Bridge $m + n$



Bridge - Estrutura

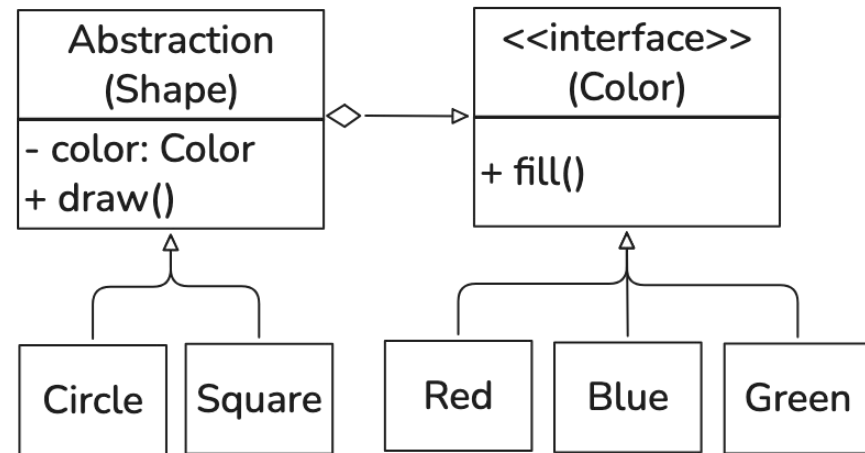
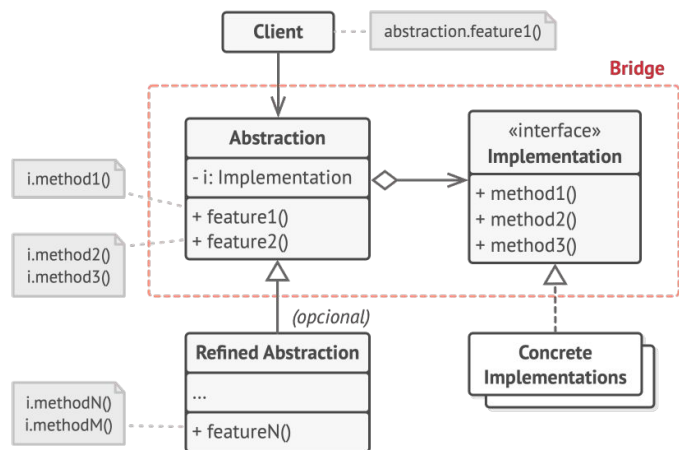
1. A **Abstraction** fornece a lógica de controle de alto nível
2. A **Implementation** declara a interface que é comum para todas as implementações
3. Já **ConcreteImplementations** contém código específico
4. E **RefinedAbstraction** fornece variantes para controle de lógica e são opcionais
5. Por fim, o **Client** está interessado apenas na **Abstraction**





Aplicando ao nosso domínio

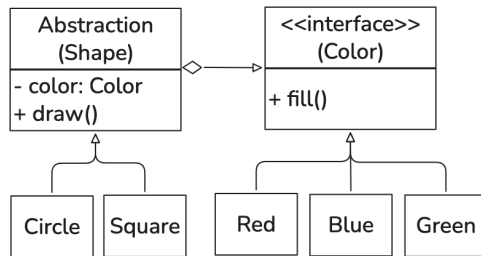
- A "ponte" é a composição entre os dois lados.





Bridge - Qual o resultado esperado?

```
1 from abc import ABC, abstractmethod
2
3 class Color(ABC):
4     @abstractmethod
5     def fill(self) -> str: ...
6
7 class Red(Color):
8     def fill(self) -> str: return "red"
9
10 class Blue(Color):
11     def fill(self) -> str: return "blue"
```



```
1 class Shape(ABC):
2     def __init__(self, color: Color):
3         self.color = color
4
5     @abstractmethod
6     def draw(self): ...
7
8 class Circle(Shape):
9     def draw(self):
10         print(f"Desenhando círculo {self.color.fill()}")
11
12 class Square(Shape):
13     def draw(self):
14         print(f"Desenhando quadrado {self.color.fill()}")
```

```
1 Circle(Red()).draw()
2 Square(Blue()).draw()
```



O Problema

- Trocar a cor não exige criar uma nova classe
 - `circle = Circle(Blue())`
- Para adicionar uma nova cor é só estender o que já existe



```
1 class Green(Color):  
2     def fill(self):  
3         return "green"
```

- Precisamos modificar Circle ou Square?



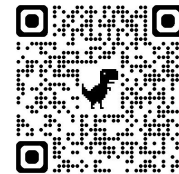
Prós e Contras do Bridge

Prós

- Dois eixos evoluem de forma independente
- Cliente só vê a abstração de alto nível
- A implementação pode ser trocada em runtime
- O **O** e **S** do SOLID são preservados

Contras

- Aplicar o padrão a uma classe altamente coesa só adiciona indireção sem ganho
- Se não existem duas dimensões reais de variação, não use Bridge.



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br